🚀 Selecting a Premium Interview Story: Stretch Goals

Only explore these stretch goals once your core checklist is passing and your `EVIDENCE.md` ledger is locked. A single, fully implemented stretch goal provides a compelling technical story for your engineering interviews.

Here are the optimal architectural blueprints for the top three stretch goals, using production-grade libraries that integrate cleanly into your existing SQLite and `google-genai` codebase.

🎨 Stretch Goal A: Fallback Image Generation (`generateImageForContent.ts` Tie-in)

When the Mismatch Guard returns `"No confident match available"`, a production-grade system shouldn't leave the content editor empty-handed. Instead, it hooks into an image generation workflow to programmatically create a precise, tailored asset.

Using your toolset, you can implement this clean, modular generation fallback:

python

```python
import os
from google import genai

class FallbackGenerationEngine:
    def __init__(self, gemini_api_key: str):
        # Initializing the modern Google GenAI Client
        self.client = genai.Client(api_key=gemini_api_key)

    def generate_fallback_asset(self, post_text: str, target_output_path: str) -> dict:
        """
        Triggered dynamically when the Mismatch Guard issues a hard rejection.
        Uses Imagen 3 to generate a missing visual match.
        """
        print(f"🔮 Mismatch Guard Fallback Active. Engineering prompt for Imagen 3...")

        # 1. Transform raw long-form blog text into a crisp, descriptive image prompt
        prompt_refinement = self.client.models.generate_content(
            model='gemini-2.5-flash',
            contents=f"Convert this blog article snippet into a short, high-fidelity photo
        )
```

```
        refined_prompt = prompt_refinement.text.strip()

        print(f"🎨 Executing Generation Prompt: {refined_prompt}")

        # 2. Invoke Imagen 3 to construct the visual asset
        result = self.client.models.generate_images(
            model='imagen-3.0-generate-002',
            prompt=refined_prompt,
            config=dict(
                number_of_images=1,
                output_mime_type="image/jpeg",
                aspect_ratio="16:9"
            )
        )

        # 3. Commit the generated byte stream to your local assets directory
        for generated_image in result.generated_images:
            with open(target_output_path, "wb") as f:
                f.write(generated_image.image.image_bytes)

        print(f"💾 Fallback asset written to storage: {target_output_path}")
        return {
            "status": "FALLBACK_GENERATED",
            "file_path": target_output_path,
            "prompt_used": refined_prompt
        }
```

Use code with caution.

🔍 Stretch Goal B: Near-Duplicate Asset Detection (Perceptual Hashing)

Uploading duplicate or minor variations of the same asset wastes storage and clutters ingestion workers. Implementing **Perceptual Hashing (pHash)** allows you to identify near-identical images by analyzing visual structure rather than fragile file metadata.

- **How to implement for $0:** Add the local `imagehash` python package to your `requirements.txt`. It runs completely on local hardware without incurring API costs.

python

```python
from PIL import Image
import imagehash
import sqlite3

def check_for_near_duplicates(incoming_image_path: str, db_file: str, hamming_threshold: i
    """
    Computes a perceptual hash of an incoming asset and compares it against
    pre-computed hashes stored in your local database.
    Triggers an early exit if a near-duplicate is detected.
    """
    # 1. Calculate the structural perceptual hash
    img = Image.open(incoming_image_path)
    current_hash = imagehash.phash(img)
    current_hash_str = str(current_hash)

    # 2. Query existing asset hashes from the local database
    with sqlite3.connect(db_file) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT image_id, file_path, phash_str FROM images WHERE phash_str
        existing_assets = cursor.fetchall()

    # 3. Evaluate the Hamming Distance (the number of bit differences)
    for img_id, file_path, stored_hash_str in existing_assets:
        stored_hash = imagehash.hex_to_hash(stored_hash_str)
        distance = current_hash - stored_hash # Overloaded operator calculates distance

        if distance <= hamming_threshold:
            print(f"🛑 Duplicate asset rejected! Close match detected with {file_path} (D:
            return True # Duplicate found

    return False # Clear to ingest
```

Use code with caution.

♿ Stretch Goal C: Automated WCAG Compliance Alt Text Generator

This is a highly practical features for modern content platforms, ensuring that every asset automatically receives rich, descriptive accessibility tags before it hits the live staging server.

- **The Strategy:** Extend your initial Phase 1 Gemini JSON Schema configuration block to require an `alt_text` field alongside the standard subject and category tags.

```json
"properties": {
    "subject": {"type": "STRING"},
    "category": {"type": "STRING"},
    "alt_text": {
        "type": "STRING",
        "description": "A high-quality WCAG-compliant descriptive alt string for visually
    }
}
```

Use code with caution.

💡 Documenting a Stretch Goal in your Build Log

To document these additions for your final technical interview presentation, update your project log with a detailed entry:

```markdown
### [Stretch Goal Integration] Deployment of Image Generation Fallback Architecture
- **What I Did:** Extended the core engine with a fallback layer that triggers when the Mi
- **AI Tool Assistance:** Assisted in setting up byte stream file handlers to capture imag
- **Interview Takeaway Point:** Instead of letting the application fail silently, the arch
```

Use code with caution.

If you are ready to wrap up your final implementation, let me know:

- Draft the comprehensive `README.md` file, complete with architecture explanation notes
- Build the automated **PyTest suite** to verify your schema parsing and mismatch rejection logic